ASSIGNMENT 1 (CO1)

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Namma Metro Smart-Card Fare System | Total Marks: 50 | Bloom's Level: K5 – Evaluate

Name: **B. Chaithanya Prakash Reddy**
Register Number: **192425255**
Course code: DSA0104 Assignment1

Task 1 – Understand the Problem and Identify the Objects

Objects, data and behaviour

- SmartCard: data is the balance in paise (private, long long). Behaviour is canTravel(fare), deduct(fare), recharge(amount) and balance(). The card owns the money, so it is the only code allowed to change it. This is how a negative balance is prevented.
- Passenger: data is the passenger type (Regular, Student, Senior) and the card. Behaviour is concession(). It is separate from the card because a person's category does not depend on the card balance.
- Journey: data is the zone, the hour and a weekend flag. Behaviour is isPeak(). Peak is a property of the journey time, not of the card.
- Zone: an enum class with Inner, Middle and Outer. A fixed set of named values that cannot be mixed with plain integers.
- FareTable: holds the base fares in paise and the surcharge percent, and gives fare(zone, peak). All fare values live in one place.

Procedural versus object-oriented style

In a procedural program the balance, fares and flags are global variables and free functions such as fare(), canTravel() and pay() read and write them directly. Any function can change the balance, so the rule that Rs 50 must remain has to be rechecked everywhere the balance is touched. In the object-oriented version the data is private inside SmartCard and the rule is enforced in one method, deduct(). The fare values move into FareTable, so a fare change touches one class instead of many call sites.

OOP principles in this system

- Encapsulation: SmartCard keeps balancePaise_ private; only deduct() and recharge() modify it.
- Abstraction: the gate calls card.canTravel(fare) without knowing how the minimum balance is checked.
- Inheritance: Passenger can be a base class with Student and Senior derived classes.
- Polymorphism: a virtual concession() lets the gate treat every passenger type through a base-class reference.

Rules every correct journey must satisfy

- R1: the balance after paying must be at least Rs 50 (5000 paise).
- R2: the fare depends on the farthest zone travelled: Rs 10, Rs 20, Rs 30.
- R3: peak hours are 8:00 to 9:59 and 17:00 to 19:59. The start is inclusive and the end exclusive, so 10:00 and 20:00 are off-peak.
- R4: the peak surcharge is exactly 20%, computed in integer paise, so there is no rounding error.
- R5: a recharge must be positive and the balance must never be negative.
- R6: an unknown zone or invalid hour is rejected and nothing is deducted.

Assumptions: hours are 0 to 23, and 20% of each base fare (200, 400, 600 paise) is a whole number of paise, so integer arithmetic is exact.

Alternative Concept and Complexity Analysis

Procedural alternative: a global long long balance with free functions fare(), canTravel() and deduct(). Time per journey is O(1) and memory per card is O(1) in both styles, so complexity does not change. The difference is maintenance. When the minimum balance rule changes in the object-oriented design, only SmartCard::canTravel changes (one place). In the procedural design every function that touches balance must be found and checked, and one missed place breaks rule R1.

Task 2 – Tokens, Keywords, Identifiers and Constants

The five kinds of tokens

- Keywords: enum, class, constexpr, const, if, else, switch, case, return, for, bool.
- Identifiers: fare, canTravel, BASE_PAISE, balance, Trip.
- Constants (literals): 1000, 20, 5000, true, '\n', "Denied (low balance)".
- Operators: +, -, *, /, %, <, >=, &&, ||, ::, <<, +=, ?:.
- Punctuators: { } ( ) [ ] ; , :.

#define, const and constexpr

#define is text substitution by the preprocessor. It has no type, no scope and no debugger symbol, and it is unsafe inside expressions unless bracketed. const creates a typed read-only object, but its value may be known only at run time. constexpr guarantees that the value is known at compile time, so it can size arrays, appear in static_assert and be folded into the code. That is why constexpr is preferred for the fare table.

enum and enum class

A plain enum leaks its names into the surrounding scope and converts silently to int, so Zone::Inner and a random integer 7 can be mixed. An enum class has its own scope (Zone::Inner) and does not convert implicitly, so wrong use becomes a compile error.

Single place for all constants

```
enum class Zone { Inner, Middle, Outer };

constexpr int       BASE_PAISE[]          = {1000, 2000, 3000};
constexpr int       PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE     = 5000;
```

Naming conventions

Identifiers use letters, digits and underscore, cannot start with a digit and cannot be keywords. Convention used here: UPPER_SNAKE_CASE for constants, camelCase for functions and variables, and PascalCase for types.

Alternative Concept and Complexity Analysis

With #define FARE_INNER 1000 and #define PEAK 20 the preprocessor substitutes text before compilation. Type checking is lost, the name is visible everywhere after the definition, and an expression such as BASE * PEAK / 100 can expand wrongly without brackets. A macro occupies no storage, and a constexpr value usually does not either, because the compiler folds it into the code. Running time is unaffected in both cases. The difference is safety, not speed.

Task 3 – Data Types and Type Compatibility

Data types chosen

- Fare: int in paise (maximum 6000), 4 bytes.
- Card balance and day totals: long long in paise, 8 bytes.
- Hour: int (0 to 23), 4 bytes.
- Peak and weekend flags: bool, 1 byte.
- Zone: enum class Zone.
- Journey count: long long, 8 bytes.

Measured sizes and ranges on my system (g++ 13.3.0)

bool 1, char 1, int 4, unsigned 4, float 4, double 8, long long 8
int range -2147483648 to 2147483647
long long range -9223372036854775808 to 9223372036854775807

Why money is stored in integer paise

Decimal fractions such as 0.1 have no exact binary form, so double sums drift. Adding 0.1 ten times gave 0.99999999999999989, and the test (s == 1.0) was false. Integer paise are exact, comparisons with == are reliable, and the minimum-balance check cannot be fooled by a tiny error.

Measured growth of rounding error (adding Rs 0.10 n times)

n=20000 double Rs 1999.9999999993 exact Rs 2000.00 error -7.235e-10
n=500000 double Rs 49999.9999995529 exact Rs 50000.00 error -4.471e-07
n=3000000 double Rs 300000.0000019568 exact Rs 300000.00 error 1.957e-06

The error grows as more values are added. In this system the base fares and 20% surcharges are whole rupees (10.00, 12.00, 36.00), which are exact in double, so the fare totals in Task 10 showed no difference. Any concession or discount that produces fractional rupees would drift, so integer paise is the safe choice.

Is int enough for a day's collection?

INT_MAX is 2,147,483,647 paise (about Rs 21.5 million). The worst case is 3,000,000 journeys × 3600 paise = 10,800,000,000 paise, about five times INT_MAX, so the network total needs long long.

Type compatibility

In a mixed expression the compiler converts to a common type: bool and char promote to int, int converts to double when mixed with double, and a signed int compared with an unsigned value is converted to unsigned.

Alternative Concept and Complexity Analysis

If money were stored as double rupees, each addition is still O(1) and each value still 8 bytes, the same as long long. The cost is accuracy: the measured error above grows from 7×10^-10 at 20,000 additions to 2×10^-6 at 3,000,000, and a balance check such as balance − fare >= 50.0 can fail by a tiny amount when the true value is exactly 50.

Task 4 – Control Implicit Conversions

Where conversions happen

- int / int is integer division: 7 / 2 gives 3, and storing it in a double gives 3.0, not 3.5.
- bool, char and short are promoted to int in arithmetic and when added to numbers.
- A signed value compared with an unsigned value is converted to unsigned; -1 becomes a very large number. The compiler warns with -Wall -Wextra.
- Narrowing: int x = 3.7; silently gives 3, but int x{3.7}; is a compile error.
- static_cast is used when the conversion is intended, for example static_cast<double>(a) / b.

Predict the Output

```
#include <iostream>

int main() {
   int a = 7, b = 2; double c = a / b;
   double d = static_cast<double>(a) / b; std::cout << c << ' ' << d << '\n';

   char ch = 'A' + 2; std::cout << ch << ' ' << ch + 1 << '\n';

   bool flag = 5; std::cout << flag << ' ' << std::boolalpha << flag << '\n';

   unsigned int u = 1; int s = -1; std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (written before running): _____

Actual output

```
3 3.5
C 68
1 true
s >= u
```

- Line 1: a / b is integer division, so 7 / 2 = 3, which is converted to 3.0 and printed as 3. The cast makes 7.0 / 2 = 3.5.
- Line 2: 'A' + 2 is int 67, stored into char gives 'C'. ch + 1 is char promoted to int, so 68 is printed instead of a character.
- Line 3: 5 converts to bool true (any nonzero value), printed as 1. After std::boolalpha it prints true.
- Line 4: s is converted to unsigned, so -1 becomes 4294967295, which is greater than 1. So s < u is false and "s >= u" is printed. The compiler warning was: comparison of integer expressions of different signedness [-Wsign-compare].

Corrected code

```
int a{7}, b{2};
double c{static_cast<double>(a) / b};              // 3.5
if (static_cast<long long>(s) < static_cast<long long>(u)) { }   // true, no sign surprise
// int x{3.7};   // compile error: narrowing conversion is caught
```

```
$ g++ -std=c++17 -Wall -Wextra t4.cpp -o t4 && ./t4
3 3.5
C 68
1 true
s >= u

$ g++ -std=c++17 -Wall -Wextra t5.cpp -o t5 && ./t5
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Figure 1: Output of the Task 4 and Task 5 programs

Alternative Concept and Complexity Analysis

Writing every conversion with static_cast and brace initialisation does not change running time or memory, because the generated machine code is the same. What changes is where mistakes are caught: narrowing conversions become compile-time errors, and a reader can see every place a value changes type.

Task 5 – Expressions and Precedence

Required expressions

```
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
long long f = base + (peak ? base * PEAK_SURCHARGE_PERCENT / 100 : 0);
```

- Relational operators (>=, <) bind tighter than &&, which binds tighter than ||. The version without brackets is therefore equivalent, but the bracketed one is easier to read and safer to edit.
- The conditional operator ?: has low precedence and is right-associative, so it is bracketed inside the sum.
- x++ gives the old value and then increments; ++x increments first and gives the new value. i = i++ changes i twice without sequencing, so it is a mistake.
- Since C++11, / truncates toward zero and (a / b) * b + a % b == a, so -10 / 3 = -3 and -10 % 3 = -1.

Predict the Output

```
int main() {
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';
    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
    int k = 0; for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
```

```
}
```

My predicted output (written before running): _____

Actual output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

- Line 1: * is applied before +, so 5 + 3*2 = 11; the brackets force (5+3)*2 = 16.
- Line 2: x++ gives 5 and then x becomes 6; ++y makes y 4 and gives 4. So z = 5 + 4 = 9, x = 6, y = 4.
- Line 3: x > y is true so the ?: gives x = 6. x % 4 * 2 is evaluated left to right: (6 % 4) * 2 = 2 * 2 = 4.
- Line 4: 10 / 3 = 3, 10 % 3 = 1, -10 / 3 = -3 (truncation toward zero), -10 % 3 = -1 (sign of the left operand).
- Line 5: i takes 0, 3, 6, 9 and k = 0 + 3 + 6 + 9 = 18; the loop stops when i = 12.

Alternative Concept and Complexity Analysis

A lookup table constexpr bool PEAK[24] indexed by the hour replaces up to four comparisons by one array access. Both are O(1). The table needs 24 bytes, is easy to edit hour by hour, but needs a range check (0 to 23) before indexing.

Task 6 – Control Structures

Design choices

- if–else for range conditions such as peak hours and balance checks.
- switch for a discrete enum such as the zone; it is clear and the compiler can warn about unhandled enum values.
- The conditional operator ?: for a short value choice such as the surcharge amount.
- A range-based for loop is best for processing the journeys because it cannot go out of bounds. A classic for loop is used when the index matters, and a while loop when reading until the input ends.
- continue skips an invalid journey; break leaves a loop or ends a switch case. If no case matches and there is no default, the switch does nothing, so a default is always written.

Pseudocode

```
ALGORITHM ComputeFare(zone, peak)
  INPUT  : zone (Inner, Middle, Outer), peak (true or false)
  OUTPUT : fare in paise, or -1 for an invalid zone
  BEGIN
    SELECT zone
      CASE Inner  : base <- 1000
      CASE Middle : base <- 2000
      CASE Outer  : base <- 3000
      DEFAULT     : RETURN -1
    END SELECT
```

```
  IF peak THEN base <- base + base * 20 / 100
  RETURN base
 END
```

Complete the Code: TODO 1 and TODO 2

```
long long fare(Zone z, bool peak) {
   long long base = 0;
   switch (z) {
      case Zone::Inner:  base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer:  base = BASE_PAISE[2]; break;
      default: return -1;
   }
   if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
   return base;
}

bool canTravel(long long balance, long long f) {
   return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
```

Actual output (matches the expected output)

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

The peak rule isPeak(hour) = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) was tested at the boundaries: hour 8 is peak, hour 10 is off-peak, hour 17 is peak, hour 20 is off-peak.

```
$ g++ -std=c++17 -Wall -Wextra t6.cpp -o t6 && ./t6
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

$ ./t6b   # boundary hours
hour 7 off-peak
hour 8 peak
hour 9 peak
hour 10 off-peak
hour 17 peak
hour 19 peak
hour 20 off-peak
```

Figure 2: Output of the Task 6 program and the boundary-hour check

Alternative Concept and Complexity Analysis

If the base fare were looked up from constexpr long long FARE[] = {1000, 2000, 3000} indexed by the zone, the time per lookup is O(1) in both (a switch may compile to a jump table). The array uses 3 × 8 = 24 bytes. A fourth zone needs one more array element for the array and one more case for the switch, but the array must be indexed with a checked value.

Task 7 – From Procedural to Object-Oriented

Procedural version

```
long long balance = 10000;                // global: any code can change it
bool canTravel(long long f) { return balance - f >= 5000; }
void pay(long long f) { balance -= f; }    // no check: the caller must remember
```

Object-oriented version

```
class SmartCard {
   long long balancePaise_;
   static constexpr long long MIN_BALANCE_PAISE = 5000;
public:
   explicit SmartCard(long long b) : balancePaise_{b} {}
   bool canTravel(long long f) const { return f >= 0 && balancePaise_ - f >= MIN_BALANCE_PAISE; }
   bool deduct(long long f) { if (!canTravel(f)) return false; balancePaise_ -= f; return true; }
   bool recharge(long long a) { if (a <= 0) return false; balancePaise_ += a; return true; }
   long long balance() const { return balancePaise_; }
};
```

- The class prevents a negative balance because balancePaise_ is private and deduct() refuses any change that would break the minimum.
- When the minimum balance changes, only one constant inside SmartCard changes. In the procedural version every place that subtracts from balance must be reviewed. The object-oriented version is therefore easier to change.

Alternative Concept and Complexity Analysis

If the balance were a public data member, time and space do not change (O(1), 8 bytes). But any code could write card.balance = -500 or skip canTravel, so rules R1 and R5 can be broken from anywhere in the program and the class cannot defend them.

Task 8 – Read and Print Data Safely

```
bool readInt(const char* prompt, int& out) {
   std::cout << prompt;
   if (!(std::cin >> out)) {
      std::cin.clear();                                    // reset error flags
      std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  // discard bad line
      return false;
   }
   return true;
}
void printMoney(long long paise) {
   std::cout << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0')
          << paise % 100 << std::setfill(' ');
}
```

- When cin >> int meets letters, failbit is set and every later read fails until cin.clear() is called. The bad characters stay in the buffer until cin.ignore() removes them.
- Money is printed from integer paise: rupees = paise / 100 and paise = paise % 100 with setw(2) and setfill('0'), so there is no rounding. fixed and setprecision(2) are used only when printing a double.

Tested with the input: abc, 25, 19

```
Enter hour (0-23):
Invalid hour, try again.      <- letters "abc" rejected, stream recovered
Enter hour (0-23):
Invalid hour, try again.      <- 25 is out of range
Enter hour (0-23):
Journey report
Zone      Outer
Hour      19
Peak      yes
Fare      Rs 36.00
```

Alternative Concept and Complexity Analysis

If each line were read with std::getline() and converted with std::stoi() inside try/catch, the stream never enters a fail state. Processing a line of length L is O(L) in both designs, and getline needs a std::string buffer of O(L) space. stoi throws std::invalid_argument or std::out_of_range, caught for each line. Note that stoi accepts "12abc" as 12 unless the position argument is checked.

Task 9 – Implement and Test the Fare Logic

Every case below was run with a table-driven test program. The expected values were calculated by hand and all cases passed (ALL PASSED).

ID	Case (zone, hour, balance)	Expected fare	Result	Status
T1–T3	Inner / Middle / Outer at 12:00, balance 10000	1000 / 2000 / 3000	Allowed	PASS
T4–T6	Inner 9:00, Middle 18:00, Outer 17:00 (peak)	1200 / 2400 / 3600	Allowed	PASS
B1–B4	Inner at hours 8, 10, 17, 20	1200 / 1000 / 1200 / 1000	Allowed	PASS
B5	Inner 12:00, balance 6000 (exactly Rs 50 left)	1000	Allowed	PASS
B6	Inner 12:00, balance 5999	1000	Denied	PASS
I1	Unknown zone (cast 7)	-1	Rejected	PASS
I4	Hour 25	-1	Rejected	PASS

Invalid input: a negative recharge is rejected by recharge() (amount <= 0), and letters typed for the hour are handled by the cin.fail() check shown in Task 8.

Five journeys calculated by hand

Start with Rs 200.00 (20000 paise). Inner 9:00 peak = 1200. Middle 12:00 = 2000. Outer 18:00 peak = 3000 + 600 = 3600. Inner 20:00 off-peak = 1000. Middle 8:00 peak = 2000 + 400 = 2400. Total = 1200 + 2000 + 3600 + 1000 + 2400 = 10,200 paise (Rs 102.00). Final balance =

9,800 paise (Rs 98.00). The program printed: 5 journeys total 10200 balance 9800, which agrees.

Alternative Concept and Complexity Analysis

A table of test cases run in a loop costs O(t) time for t tests and O(t) space for the table. A new boundary case is one new row instead of a new block of code, so it is easier to extend and review than separate hand-written checks.

Task 10 – Compare Expectations with Execution

Fares cycled through 1000, 2000 and 3000 paise. The exact paise total was compared with the same fares accumulated as double rupees and with an int total.

```
20000   paise=39999000   double(paise)=39999000.00   diff=0.00   int=39999000
500000  paise=999999000  double(paise)=999999000.00  diff=0.00   int=999999000
3000000 paise=6000000000 double(paise)=6000000000.00 diff=0.00   int=1705032704
```

- Expected: the double and paise totals agree, because these fares are whole rupees and exact in binary. Observed: the difference was 0.00 at all three sizes.
- Expected: int overflows at 3,000,000 journeys because 6,000,000,000 is greater than 2,147,483,647. Observed: the int total printed 1705032704, which is 6,000,000,000 minus 4,294,967,296. At 20,000 and 500,000 journeys int was correct.
- Compiler warnings with -Wall -Wextra: the only warning in the programs was -Wsign-compare in the Task 4 program (comparing int with unsigned int), meaning the signed value is converted to unsigned and may give a wrong result. All other programs compiled with no warnings.

```
$ g++ -std=c++17 -Wall -Wextra t10.cpp -o t10 && ./t10
20000 paise=39999000 double(paise)=39999000.00 diff=0.00 int=39999000
500000 paise=999999000 double(paise)=999999000.00 diff=0.00 int=999999000
3000000 paise=6000000000 double(paise)=6000000000.00 diff=0.00 int=1705032704
```

Figure 3: Totals for 20,000, 500,000 and 3,000,000 journeys

Alternative Concept and Complexity Analysis

At 3,000,000 journeys an int fails by overflow (signed overflow is undefined behaviour; the value wrapped around). A double does not overflow, but it loses precision because of binary rounding of fractional values, as shown in Task 3; it would also lose whole-number precision beyond 2^53. A long long is exact up to about 9.2×10^18. Time per addition is O(1) for all three, and the space is 4 bytes for int and 8 bytes for double and long long.

Task 11 – Recommend and Defend the Final Design

- Types: long long paise for money, balances and totals (int overflowed at 3,000,000 journeys); int for the hour; bool for flags; enum class for zones.
- Constants: one constexpr block for the fares, surcharge percent, minimum balance and peak hours.
- Classes: SmartCard (private balance, with canTravel, deduct and recharge), FareTable (fare by zone and peak), and Journey (zone and hour).
- Fare revision: change one constexpr value, recompile, rerun the Task 9 test program (all cases must pass), then deploy to the gates.

- Limitations: every fare change needs a recompile and redeployment; there is no storage of card data; concessions and new rules need more classes.
- Whole network: the design stays correct for 3,000,000 journeys, because the long long total of 6,000,000,000 paise was exact in the test, while int gave a wrong value.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, the extra start-up time is O(z) for z fare values and a few bytes of storage, which is negligible compared with recompiling and redeploying after each revision. The trade-off is that the file must be validated (count, non-negative values), and a corrupt file needs a safe default or a refusal to start.

Task 12 – Day-4 Design Challenge

A. Impact Analysis

- Constants: BASE_PAISE gets a fourth value 6000, and WEEKEND_DISCOUNT_PERCENT = 10 is added.
- Enum: Airport is added to Zone.
- The switch in fare() needs a new case, and the peak, weekend and pass rules multiply the branches.
- Expression: the weekend discount must not combine with the peak surcharge, so the rule becomes weekend ? discount : peak ? surcharge : none.
- SmartCard: the balance logic is unchanged, but a pass journey must not deduct money. A new Pass class holds two zones.

Assumptions: when the weekend discount applies, the peak surcharge is not applied. A pass journey costs Rs 0 and the Rs 50 minimum does not apply to it. A journey is covered if the farthest zone is one of the two pass zones.

B. Evaluation of the Original Design

- Airport on a weekend: the original switch has no Airport case, so default returns -1 and the passenger is wrongly denied.
- Outer at 9 AM on a Saturday: the old rule gives 3000 + 600 = 3600, but the new rule requires 3000 − 300 = 2700. The old fare(zone, peak) has no weekend input and cannot express this.
- Pass holder in Inner or Middle: the old fare() returns 1000 or 2000 and the card deducts it, which breaks the pass rule.
- Conclusion: SmartCard can be retained, the switch-based fare() must be modified, and a Pass class is added.

C. Adaptation and working code

```
enum class Zone { Inner, Middle, Outer, Airport, COUNT };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000, 6000};
constexpr int PEAK_SURCHARGE_PERCENT   = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
static_assert(sizeof(BASE_PAISE) / sizeof(BASE_PAISE[0]) == static_cast<std::size_t>(Zone::COUNT),
        "fare table must match number of zones");

long long fare(Zone z, int hour, bool weekend) {
   int i = static_cast<int>(z);
```

```
    if (i < 0 || i >= static_cast<int>(Zone::COUNT)) return -1;
    long long base = BASE_PAISE[i];
    if (weekend) return base - base * WEEKEND_DISCOUNT_PERCENT / 100;   // no surcharge on weekends
    if (isPeak(hour)) return base + base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

class Pass {
    Zone a_, b_;
public:
    Pass(Zone a, Zone b) : a_{a}, b_{b} {}
    bool covers(Zone farthest) const { return farthest == a_ || farthest == b_; }
};
```

Actual output of the revised program

```
Airport weekday off-peak: 6000
Airport weekday peak: 7200
Airport weekend: 5400
Outer weekend 9AM (no surcharge): 2700
Inner weekend: 900
Middle weekend: 1800
Pass journey in Middle fare 0, balance 20000
Outer (not on pass) fare 3000, balance 17000
```

D. Evidence-Based Defence

- Decision: MODIFY, not redesign. The SmartCard balance logic was untouched, which shows the original object boundary was correct.
- The switch was replaced by a table indexed by zone, so a fifth zone costs one array value and one enum value, and static_assert stops the table and the enum from getting out of step.
- Test evidence: all revised fares matched the hand-calculated values (Airport 6000, 7200 on weekday peak, 5400 on weekends; Outer weekend 2700), and the pass journey left the balance unchanged at 20000 while the non-pass journey deducted 3000.
- Trade-off: the table needs a bounds check and the fare function will grow with each new rule. If many more rule types arrive, a FareRule class hierarchy using polymorphism would be the next step.